

SHREY GOEL

Gameplay / Engine Programmer

shrey.goel619@gmail.com | [Portfolio](#) | [LinkedIn](#) | [GitHub](#)

EDUCATION

DePaul University — M.S., Game Development, Chicago, IL Sept 2025 – June 2027 (Expected)

LANGUAGES & TECHNOLOGIES

Languages: C++, C#, C, Python, Kotlin, Java, JavaScript, SQL, Bash

Engines & Graphics: Unreal Engine (C++ & Blueprints), Unity, DirectX 11, HLSL, SIMD/SSE, Multithreading & Concurrency

Tools: Git, Perforce, PlasticSCM, .NET, Jetpack Compose, SQLDelight, Unix/Linux

DEVELOPMENT EXPERIENCE

Lead Developer — The AAT Project

March 2026 – Present

Remote · Henry's Big Heist (stealth game) · ~6,600 LOC across 13 subsystems

- Architected a single reusable vision/detection system (ISightSource interface + SightSensor.TrySpot) combining 3D field-of-view checks, raycast line-of-sight, and wall occlusion, powering every seeing hazard — guards and cameras alike — with zero duplicated detection code
- Built a data-driven guard AI as a ScriptableObject finite state machine (4 states, 5 reusable transitions), letting new behaviors ship as Inspector assets with no changes to agent code
- Designed a precognition ability that predicts enemy paths by reusing existing NavMesh data at no extra simulation cost, with per-guard difficulty injected through swappable Strategy-pattern profiles
- Standardized communication across AI, HUD, audio, hazard, and pickup subsystems through a central state machine and C# static events, removing direct cross-system references and enabling isolated testing
- Built a procedural audio layer that re-colors a single footstep clip with pitch/volume jitter and animation-synced cadence, cutting required audio assets while eliminating repetition fatigue
- Designed a player clone ability gated by a consumable charge economy tied to optional collectibles, with real-vs-clone disambiguation that eliminates an entire class of false game-over bugs

Wispy Moon Studios — Remote

Lead Developer

March 2025 – August 2025

- Led a team of 3 programmers and 5+ designers (game, level, sound) through weekly planning meetings, aligning scope, delegating tasks, and tracking milestones against client specifications
- Designed reinforcement-learning-driven agentic AI combined with behavior trees and Environment Query Systems (EQS) to produce realistic, dynamic animal behavior in response to environmental changes

Unreal Engine Programmer

September 2024 – March 2025

- Led development of core stealth gameplay mechanics and system design
- Built automated functional tests for core mechanics to catch regressions early in development

Game Developer — P1 Games

August 2024 – September 2024

Remote

- Built and optimized open-source Unity games within a 200+ person cross-functional community of artists, sound designers, and developers
- Iterated on game mechanics based on weekly playtest feedback

PROJECTS

Real-Time 3D Rendering Engine — C++ / DirectX 11 (DePaul) · [Demo](#)

Spring 2026

- Wrapped the full DirectX 11 device pipeline (device, swap chain, render targets, depth/stencil, rasterizer, blend, viewport) in 15 dedicated RAIL state classes with deleted copy semantics, eliminating manual COM Release calls and reference-count leaks
- Authored a 4,356-line SIMD-accelerated 3D math library (SSE __m128 intrinsics, 16-byte aligned) with proxy return types that defer costly operations like square roots, shipped as a versioned DLL reused across two C++ engine projects
- Designed a Template Method render contract (SetState → SetDataGPU → Draw → RestoreState) driving 6 shader variants across 7 graphics-object types with per-object constant buffers and Phong lighting, preventing GPU state leakage between draws
- Built an 873-line custom parent-child-sibling scene graph with O(1) insert/remove and forward/reverse iterators, powering hierarchical transforms for every object in the scene
- Implemented a multi-camera system (5 runtime-switchable cameras plus a free-fly debug camera) and fail-fast ABI version checks across 4 self-authored DLLs, catching library mismatches before any content loads

Real-Time Audio Engine — C++ / XAudio2 (DePaul) · [Demo](#)

Spring 2026

- Architected an 8.2K-LOC, 3-thread share-nothing actor model (game, audio, file threads) that decouples the real-time XAudio2 mixer from disk I/O and gameplay logic, eliminating audio dropouts from blocking calls
- Built a lock-managed, allocation-free 32-slot circular command buffer with power-of-two bitmask indexing for O(1) cross-thread transport of 15+ command types
- Designed a 150-entry generational handle table with salted IDs and per-slot locking to detect stale cross-thread references in O(1), converting use-after-free bugs into safe, testable results
- Implemented RAIL validity guards with all-or-nothing two-handle acquisition and rollback, eliminating torn state when binding paired resources across threads
- Built a pooled voice manager with 3 polymorphic XAudio2 callback strategies, including sample-accurate gapless buffer chaining, to drive an adaptive music system with zero game-thread involvement

Space Invaders — C# Game Framework (DePaul) · [Demo](#)

Winter 2026

- Built an engine-agnostic game framework (~11.8K LOC, 162 files) around 15+ design patterns with UML-documented architecture for scalable development and onboarding
- Designed a generic object-pooling framework (Template Method) reused across 8 subsystem managers, eliminating per-frame garbage-collection stalls via active/reserve free-lists
- Built a 4-stage collision pipeline (Composite traversal, AABB broad-phase rejection, double-dispatch Visitor resolution across 25+ object types, Observer fan-out) supporting 13 registered collision-pair types with zero-edit extensibility
- Implemented a priority-queued, Command-pattern event scheduler with time-dilation pause, driving 11 timed gameplay commands with no timer drift across scene transitions
- Applied Flyweight and Proxy patterns to render 55+ on-screen entities from a shared sprite/texture pool with 4-layer priority draw batching, cutting redundant memory use versus a per-instance asset model

Mihon Manga Reader — Android / Kotlin, Open Source · [GitHub](#)

June 2026 – Present

- Built privacy- and AI-focused features for the Mihon Android manga reader using Kotlin, Jetpack Compose, and a clean multi-module architecture
- Developed a privacy-first, on-device tracking system with SQLDelight migrations, layered domain/data architecture, and backup/restore support
- Redesigned library update scheduling to improve chapter-release detection, and prototyped AI-powered recommendations from reading history